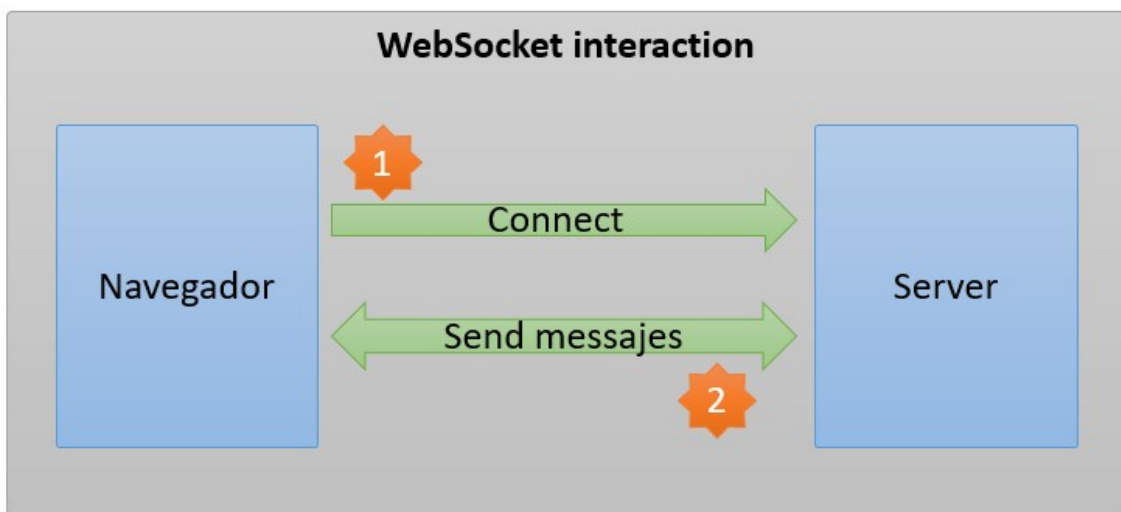


Web Sockets

WebSocket es un protocolo de comunicación que permite el intercambio de datos bidireccional y full-duplex entre un cliente y un servidor a través de una conexión persistente. Los WebSocket trabajan mediante conexiones TCP con el servidor, por lo que permite mantener una conexión activa y bidireccional entre el Servidor y el Navegador. Para que una conexión WebSocket se dé, debemos de tener dos partes, un WebSocket Server y un WebSocket Cliente, El Server es aquel que acepta las conexiones y que representa el BackEnd y el cliente en este caso sería el navegador. En este contexto, el cliente es el que establece inicialmente la conexión con el servidor. Una vez establecida la conexión, tanto el servidor como el cliente se podrá enviar mensajes simultáneamente. Veamos la siguiente imagen:



Imagen

CONSTRUYENDO UNA APLICACIÓN USANDO WEBSOCKETS

Configuración previa

Crear una carpeta de trabajo llamada CODE-WS, crear el archivo main.go

Configuración del servidor HTTP

Dado que los websockets se basan en HTTP, nuestro primer paso sería crear un servidor HTTP básico que pueda aceptar conexiones de clientes y servir mensajes.

```
func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        http.ServeFile(w, r, "websockets.html")
    })
    http.HandleFunc("/ws", handleConnections)
    http.HandleFunc("/api/message", handleAPI)

    go handleMessages()

    fmt.Println("Servidor escuchando en :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Iniciar el handshake

Para configurar una conexión WebSocket, se requiere un protocolo de enlace único entre el cliente y el servidor. Este protocolo utiliza el método Upgrade() para actualizar la conexión del servidor HTTP al protocolo WebSocket.

A continuación, se muestra la implementación de la función para Upgrade() la biblioteca github.com/gorilla/websocket

```
package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "sync"

    "github.com/gorilla/websocket"
)

type Message struct {
    Content string `json:"content"`
}

var (
    clients    = make(map[*websocket.Conn]bool)
    broadcast  = make(chan Message)
    upgrader   = websocket.Upgrader{
        CheckOrigin: func(r *http.Request) bool { return true },
    }
    mu sync.Mutex
)

func upgrade(w http.ResponseWriter, r *http.Request) {
    c, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        log.Println("websocket upgrade failed:", err)
        return
    }
    clients[c] = true
    log.Println("New websocket connection")
}
```

```
func handleConnections(w http.ResponseWriter, r *http.Request) {  
    ws, err := upgrader.Upgrade(w, r, nil)  
    if err != nil {  
        log.Printf("Error al actualizar a WebSocket: %v\n", err)  
        return  
    }  
    defer ws.Close()  
  
    mu.Lock()  
    clients[ws] = true  
    mu.Unlock()  
  
    for {  
        var msg Message  
        err := ws.ReadJSON(&msg)  
        if err != nil {  
            mu.Lock()  
            delete(clients, ws)  
            mu.Unlock()  
            break  
        }  
    }  
}
```

Envío y recepción de mensajes mediante websockets

A continuación, se muestra el controlador utilizado para recibir y enviar mensajes a la Web cliente. La función "handleAPI" recibe el request de la petición HTTP del endpoint API y envía el mensaje recibido a través del canal broadcast para el WebSocket. En la función "handleMessages", dentro del bucle se recibe el mensaje desde el canal broadcast, se usa el bloqueo Lock() para exclusión mutua y se recorre el arreglo de los canales WebSocket (clientes) para enviar el mensaje recibido a las Web cliente.

```
func handleAPI(w http.ResponseWriter, r *http.Request) {  
    var msg Message  
    err := json.NewDecoder(r.Body).Decode(&msg)  
    if err != nil {  
        http.Error(w, "Error al decodificar el mensaje", http.StatusBadRequest)  
        return  
    }  
  
    broadcast <- msg  
    w.WriteHeader(http.StatusCreated)  
    json.NewEncoder(w).Encode(msg)  
}  
  
func handleMessages() {  
    for {  
        msg := <-broadcast  
        mu.Lock()  
        for client := range clients {  
            err := client.WriteJSON(msg)  
            if err != nil {  
                log.Printf("Error al enviar mensaje: %v", err)  
                client.Close()  
                delete(clients, client)  
            }  
        }  
        mu.Unlock()  
    }  
}
```

Construimos el frontend de mensajes

En la carpeta de trabajo, crear el archivo websockets.html con el siguiente contenido para recibir los mensajes que son enviados por el WebSocket:

```
websockets.html > html > script
1  <html>
2    <div>
3      <h1>Go + websockets</h1>
4      <pre id="output"></pre>
5    </div>
6    <style>
7      html {
8        text-align: center;
9        font-size: 16px;
10     }
11     div {
12       padding: 1rem;
13     }
14     #input {
15       font-size: 16px;
16     }
17     p {
18       font-size: 16px;
19     }
20   </style>
21   <script>
22     var output = document.getElementById("output");
23     var socket = new WebSocket("ws://localhost:8080/ws");
24
25     socket.onopen = function () {
26       output.innerHTML += "Status: Connected\n";
27     };
28
29     socket.onmessage = function (e) {
30       output.innerHTML += "\nServer: " + e.data + "\n";
31     };
32
33   </script>
34 </html>
```

Configurar el proyecto para usar la biblioteca github.com/gorilla/websocket, abrir en símbolo del sistema, acceder a la carpeta de trabajo y ejecutar los comandos:

```
D:\CODE-WS>go mod init code-ws
go: creating new go.mod: module code-ws
go: to add module requirements and sums:
go mod tidy
```

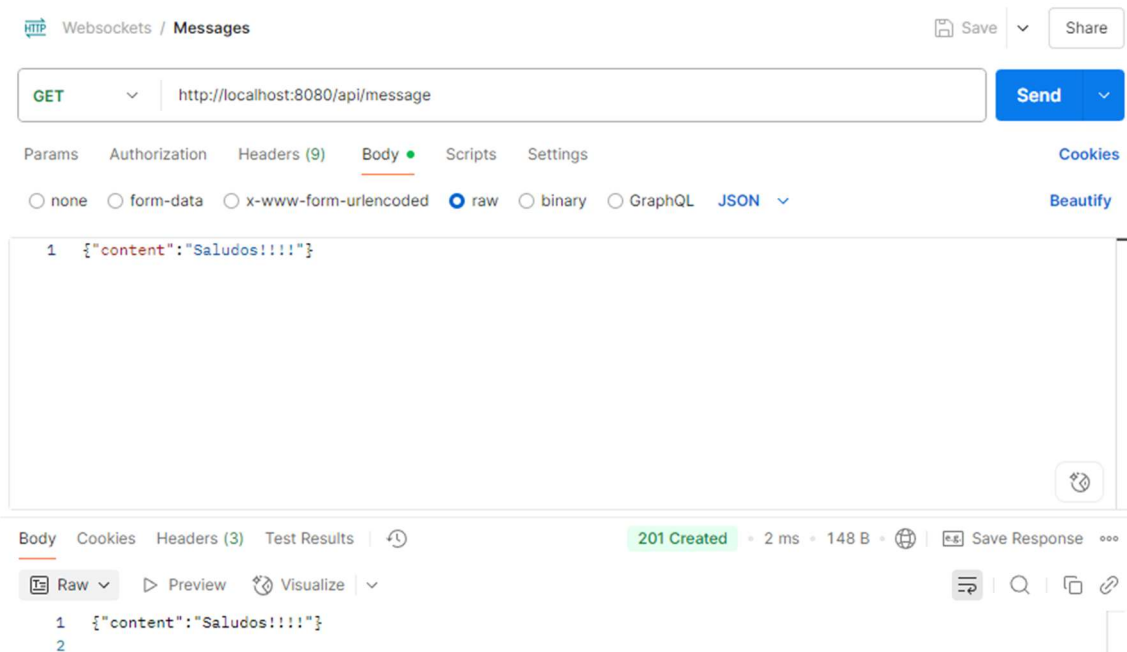
```
D:\CODE-WS>go mod tidy
go: finding module for package github.com/gorilla/websocket
go: found github.com/gorilla/websocket in github.com/gorilla/websocket v1.5.3
```

```
D:\CODE-WS>go get -u -v github.com/gorilla/websocket
```

Verificar que se crearon los archivos de módulo, go.mod y go.sum

```
CODE-WS
:GO go.mod
:GO go.sum
:GO main.go
📄 websockets.html
```

Se muestra el envío del mensaje a través de postman y es enviado al WebSocket:



Se muestra la salida en la web

